Untitled

2025-05-08

Load data

```
exp_df = read.csv("experiment_data/exp_df_real_exp.csv")
head(exp_df)
```

```
##      AI explanation        ResponseId             PROLIFIC_PID test_order
## 1 pred2        none R_7rMssTr2X2F4Tur 66045b87ffe71e6e521f357d       1938
## 2 pred2        none R_7rMssTr2X2F4Tur 66045b87ffe71e6e521f357d        229
## 3 pred2        none R_7rMssTr2X2F4Tur 66045b87ffe71e6e521f357d        988
## 4 pred2        none R_7rMssTr2X2F4Tur 66045b87ffe71e6e521f357d        679
## 5 pred2        none R_7rMssTr2X2F4Tur 66045b87ffe71e6e521f357d        841
## 6 pred2        none R_7rMssTr2X2F4Tur 66045b87ffe71e6e521f357d       2641
##   test_index test_mse Finished displayed_reward human_response hai_response
## 1        120 12488.78     True                3            236          143
## 2        121 12488.78     True                3            218          221
## 3        122 12488.78     True                3            309          212
## 4        123 12488.78     True                3            208          113
## 5        124 12488.78     True                3            305          159
## 6        125 12488.78     True                3            279          160
##   Order Year_Built Gr_Liv_Area Garage_Cars Fireplaces Year_Remod Overall_Qual
## 1  1938       1955        1540           1          0       1955            4
## 2   229       1996        2260           2          0       1997            7
## 3   988       1977        1567           2          2       1977            6
## 4   679       1949        1040           2          0       1950            4
## 5   841       1994        1097           2          0       1995            5
## 6  2641       1925        1641           2          1       1990            7
##   SalePrice    pred1    pred2
## 1    115000 100803.2 139084.0
## 2    328000 199864.5 217583.2
## 3    196000 189579.4 201326.0
## 4     90000 107110.1 113666.0
## 5    145000 162989.5 150749.8
## 6    178000 180625.7 154446.4
```

Filter data based on the criteria in the pre-registration

```
no_revise_prolific_pids = (exp_df %>% group_by(PROLIFIC_PID, AI, explanation) %>% summarise(a = sum(huma
```

```
## `summarise()` has grouped output by 'PROLIFIC_PID', 'AI'. You can override
## using the `.groups` argument.
```

```
consecutive_human_response = c(1)
for (i in 2:length(exp_df$human_response)) {
  if (exp_df$human_response[[i]] == exp_df$human_response[[i-1]]) {
```

```
    consecutive_human_response = c(consecutive_human_response, consecutive_human_response[i-1] + 1)
  } else {
    consecutive_human_response = c(consecutive_human_response, 1)
  }
}
exp_df$consecutive_human_response = consecutive_human_response

consecutive_hai_response = c(1)
for (i in 2:length(exp_df$hai_response)) {
  if (exp_df$hai_response[[i]] == exp_df$hai_response[[i-1]]) {
    consecutive_hai_response = c(consecutive_hai_response, consecutive_hai_response[i-1] + 1)
  } else {
    consecutive_hai_response = c(consecutive_hai_response, 1)
  }
}
exp_df$consecutive_hai_response = consecutive_hai_response
repeat_response_prolific_pids = (exp_df %>% group_by(PROLIFIC_PID, AI, explanation) %>% summarise(a = ar
```

```
## `summarise()` has grouped output by 'PROLIFIC_PID', 'AI'. You can override
## using the `.groups` argument.
```

```
high_rel_error_prolific_pid = (exp_df %>% group_by(PROLIFIC_PID, AI, explanation) %>% summarise(a = ((hu
```

```
## Warning: Returning more (or less) than 1 row per `summarise()` group was deprecated in
## dplyr 1.1.0.
## i Please use `reframe()` instead.
## i When switching from `summarise()` to `reframe()`, remember that `reframe()`
##   always returns an ungrouped data frame and adjust accordingly.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## `summarise()` has grouped output by 'PROLIFIC_PID', 'AI', 'explanation'. You
## can override using the `.groups` argument.
```

```
filtered_exp_df = exp_df |>
  filter(!(PROLIFIC_PID %in% high_rel_error_prolific_pid) & !(PROLIFIC_PID %in% repeat_response_prolifi
```

Duration

```
duration_df = read.csv("experiment_data/human-AI-house-pricing-real-exp1_November 16, 2025_18.36.csv")

filtered_response_id = unique(filtered_exp_df$ResponseId)
duration_df = duration_df %>% select(ResponseId, Duration..in.seconds., AI, explanation) %>% filter(Resp
duration_df %>% group_by(AI, explanation) %>% summarise(duration = mean(Duration..in.seconds.)/60, dura
```

```
## `summarise()` has grouped output by 'AI'. You can override using the `.groups`
## argument.
```

```
## # A tibble: 6 x 5
## # Groups:   AI [2]
##   AI    explanation duration duration25 duration75
##   <chr> <chr>          <dbl>      <dbl>      <dbl>
## 1 pred1 both            23.3       13.1       32.1
## 2 pred1 none            22.4       14.5       28.4
## 3 pred1 shap            21.3       12.0       27.9
```

```
## 4 pred2 both            22.2       11.9       28.0
## 5 pred2 none            22.5       12.7       30.5
## 6 pred2 shap            23.1       13.2       27.0
```

Anchoring on AI

```
filtered_exp_df %>%
  mutate(ai_pred = ifelse(AI == "pred1", pred1, pred2)) %>%
  mutate(distance_to_ai = abs(hai_response * 1000 - ai_pred) / SalePrice,
         distance_to_h = abs(hai_response - human_response) / SalePrice * 1000) %>%
  group_by(AI, explanation) %>%
  summarise(mean_distance_to_ai = mean(distance_to_ai),
            distance_to_ai25 = quantile(distance_to_ai, 0.25),
            distance_to_ai75 = quantile(distance_to_ai, 0.75),
            mean_distance_to_h = mean(distance_to_h),
            distance_to_h25 = quantile(distance_to_h, 0.25),
            distance_to_h75 = quantile(distance_to_h, 0.75),
            n = n()/12)
```

```
## `summarise()` has grouped output by 'AI'. You can override using the `.groups`
## argument.
```

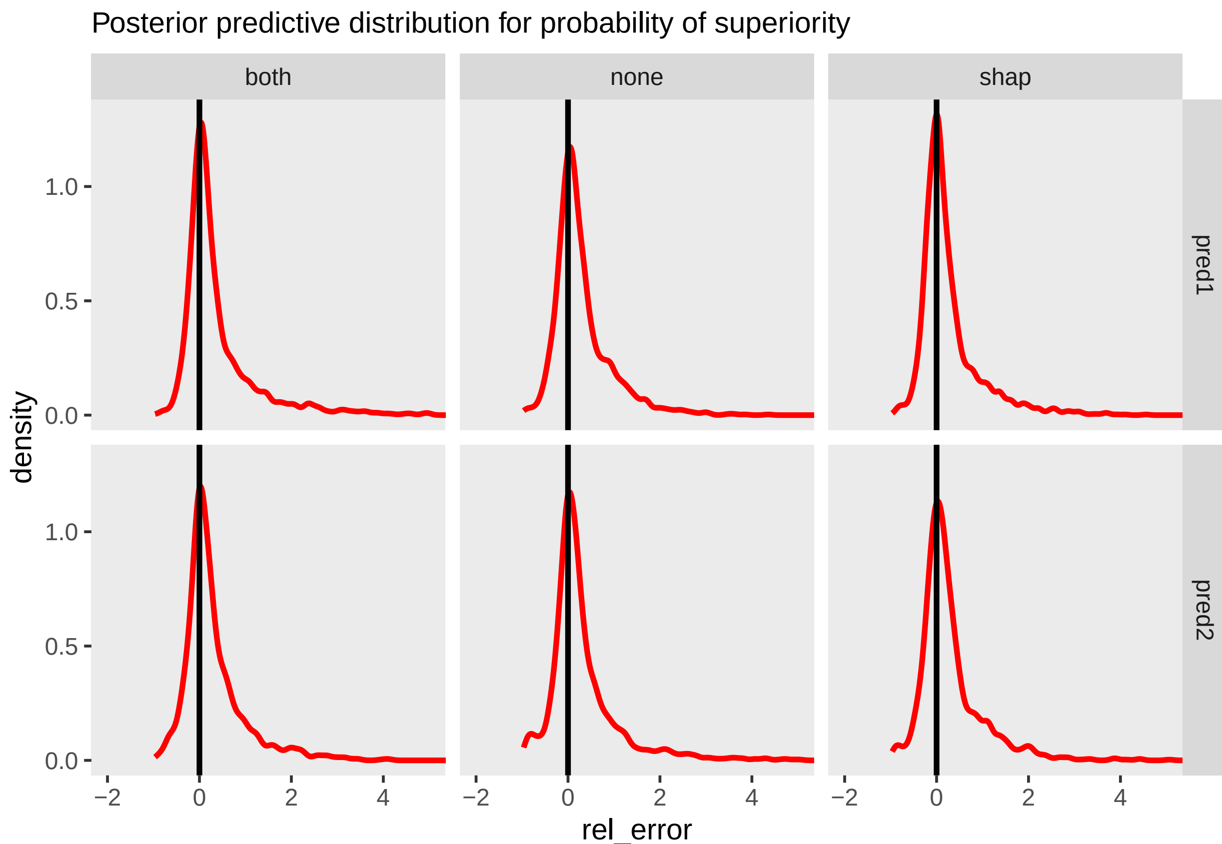
```
## # A tibble: 6 x 9
## # Groups:   AI [2]
##   AI    explanation mean_distance_to_ai distance_to_ai25 distance_to_ai75
##   <chr> <chr>                     <dbl>            <dbl>            <dbl>
## 1 pred1 both                      0.303           0.0277            0.253
## 2 pred1 none                      0.226           0.0276            0.258
## 3 pred1 shap                      0.211           0.0201            0.222
## 4 pred2 both                      0.206           0.0273            0.260
## 5 pred2 none                      0.259           0.0265            0.260
## 6 pred2 shap                      0.203           0.0215            0.224
## # i 4 more variables: mean_distance_to_h <dbl>, distance_to_h25 <dbl>,
## #   distance_to_h75 <dbl>, n <dbl>
```

Plot the distribution of relevant error

```
exp_df.mod = filtered_exp_df %>%
  pivot_longer(cols = c(human_response, hai_response), names_to = "block", values_to = "response", names
  mutate(rel_error = (response*1000 - SalePrice)/SalePrice)
  # filter(abs(rel_error) <= 5)
exp_df.mod |>
  ggplot() +
  geom_density(aes(x = rel_error), color = "red", size = 1, alpha = 1) +
  geom_vline(aes(xintercept = 0), size = 1, alpha = 1) +
  coord_cartesian(xlim = c(-2, 5)) +
  facet_grid(AI ~ explanation) +
  labs(subtitle = "Posterior predictive distribution for probability of superiority") +
  theme(panel.grid = element_blank())
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
```

```
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



Fit the model

```
priors_rel_error <-  c(
  # Priors for fixed effects of the location (rel_error)
  prior(normal(0, 1), class = b),
  prior(normal(0, 2.5), class = Intercept),

  # Priors for random effects of the location (rel_error)
  prior(cauchy(0, 1), class = sd, group = "PROLIFIC_PID"),

  # Priors for fixed effects of the scale (sigma)
  prior(normal(0, 1), class = b, dpar = "sigma"),
  prior(normal(0, 2.5), class = Intercept, dpar = "sigma"),

  # Priors for random effects of the scale (sigma)
  prior(cauchy(0, 1), class = sd, group = "PROLIFIC_PID", dpar = "sigma"),

  # Prior for the degrees of freedom (nu)
  prior(gamma(2, 0.1), class = nu)
)
```

```r
rel_error_model = brm(data = exp_df.mod,
                      formula = bf(rel_error ~ AI*explanation + block + (1| PROLIFIC_PID),
                                   sigma ~ AI*explanation + block + (1| PROLIFIC_PID)),
                 prior = priors_rel_error, family = "student",
                 iter = 20000, warmup = 8000, chains = 4, cores = 4,
                 file = "brms_model_fits/lo_rel_error_mdl")
rel_error_model
```

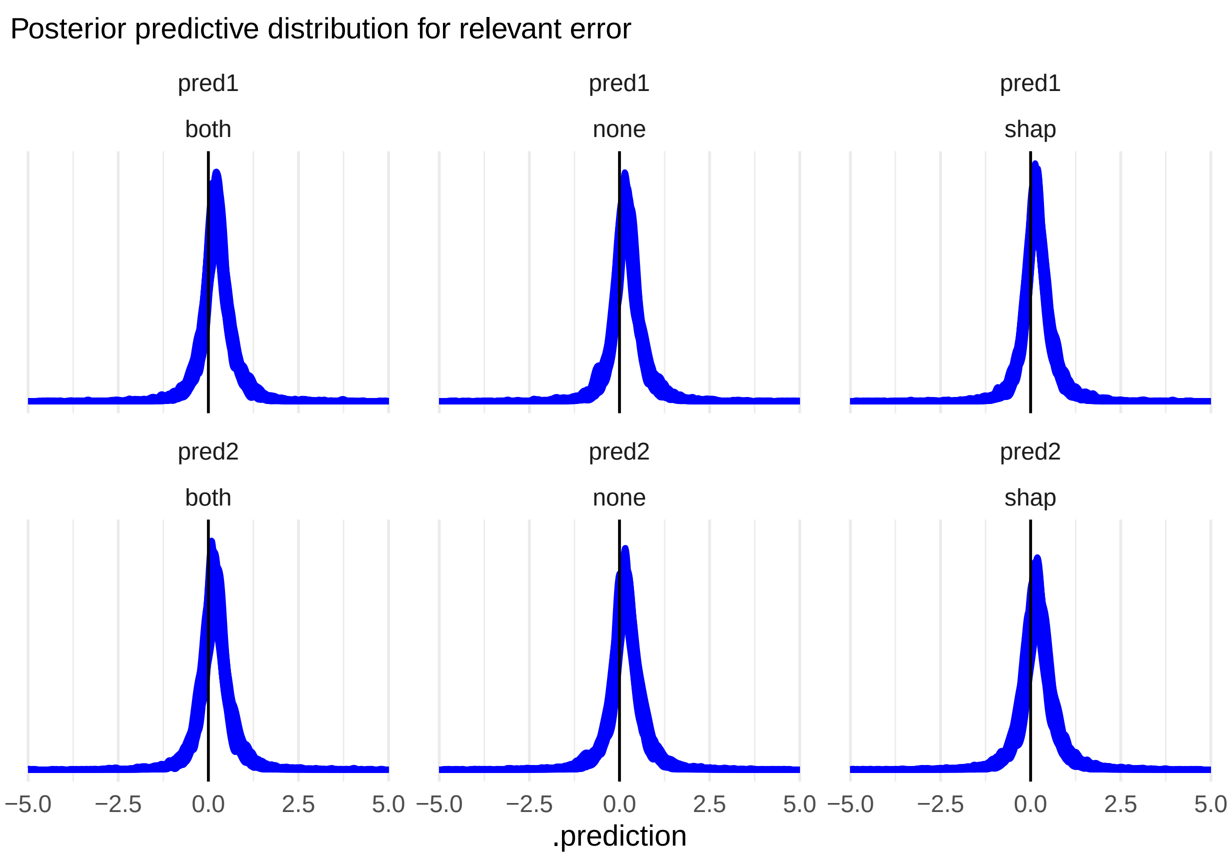
```
##  Family: student
##   Links: mu = identity; sigma = log; nu = identity
## Formula: rel_error ~ AI * explanation + block + (1 | PROLIFIC_PID)
##          sigma ~ AI * explanation + block + (1 | PROLIFIC_PID)
##    Data: exp_df.mod (Number of observations: 10104)
##   Draws: 4 chains, each with iter = 20000; warmup = 8000; thin = 1;
##          total post-warmup draws = 48000
##
## Multilevel Hyperparameters:
## ~PROLIFIC_PID (Number of levels: 421)
##                       Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(Intercept)             0.30      0.01     0.27     0.33 1.00     7122    14766
## sd(sigma_Intercept)       0.48      0.02     0.45     0.52 1.00    13521    26423
##
## Regression Coefficients:
##                                 Estimate Est.Error l-95% CI u-95% CI Rhat
## Intercept                           0.20      0.04     0.13     0.28 1.00
## sigma_Intercept                    -1.56      0.06    -1.69    -1.43 1.00
## AIpred2                            -0.07      0.05    -0.17     0.04 1.00
## explanationnone                    -0.04      0.05    -0.14     0.06 1.00
## explanationshap                    -0.08      0.05    -0.17     0.02 1.00
## blockhuman                          0.09      0.01     0.07     0.10 1.00
## AIpred2:explanationnone             0.04      0.07    -0.10     0.19 1.00
## AIpred2:explanationshap             0.07      0.07    -0.07     0.22 1.00
## sigma_AIpred2                      -0.02      0.09    -0.19     0.15 1.00
## sigma_explanationnone              -0.04      0.09    -0.21     0.14 1.00
## sigma_explanationshap              -0.05      0.08    -0.21     0.12 1.00
## sigma_blockhuman                    0.71      0.02     0.67     0.75 1.00
## sigma_AIpred2:explanationnone       0.07      0.12    -0.17     0.31 1.00
## sigma_AIpred2:explanationshap       0.11      0.12    -0.13     0.35 1.00
##                                 Bulk_ESS Tail_ESS
## Intercept                           3128     6495
## sigma_Intercept                     9079    18641
## AIpred2                             2967     6552
## explanationnone                     3288     6464
## explanationshap                     2826     6914
## blockhuman                         69220    38343
## AIpred2:explanationnone             3155     6295
## AIpred2:explanationshap             2896     7391
## sigma_AIpred2                       8560    16591
## sigma_explanationnone               8819    16713
## sigma_explanationshap               8701    16728
## sigma_blockhuman                   72160    38481
## sigma_AIpred2:explanationnone       8743    16961
## sigma_AIpred2:explanationshap       8638    16612
##
```

```
## Further Distributional Parameters:
##    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
## nu     3.93      0.18     3.59     4.30 1.00    53381    39568
##
## Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

Plot the posterior predictive distribution for relevant error

```
exp_df.mod %>%
  data_grid(AI, explanation, block, PROLIFIC_PID) |>
  add_predicted_draws(rel_error_model, ndraws = 100, re_formula = NA) %>%
  ggplot() +
  # geom_density(data = exp_df.mod, aes(x = rel_error), color = "red", size = 1, alpha = 1) +
  geom_density(aes(x = .prediction, group = .draw), color = "blue", size = 1, alpha = 0.1) +
  geom_vline(aes(xintercept = 0), size = 0.5, alpha = 1) +
  xlim(-5, 5) +
  scale_y_continuous(NULL, breaks = NULL) +
  facet_wrap(vars(AI, explanation)) +
  labs(subtitle = "Posterior predictive distribution for relevant error") +
  theme_minimal()
```

```
## Warning: Removed 96 rows containing non-finite outside the scale range
## (`stat_density()`).
```



Plot the reduction in APE

```
plot_df = filtered_exp_df %>%
  mutate(row_id = row_number(),
         block = list(c("human", "hai"))) %>%
  unnest(block) %>%
  add_predicted_draws(rel_error_model, ndraws = 500, seed = 1234, re_formula = NULL) %>%
  pivot_wider(names_from = block, values_from = .prediction) %>%
  group_by(row_id, AI, explanation) %>%
  summarise(human = mean(human, na.rm = T), hai = mean(hai, na.rm = T))
```

```
## `summarise()` has grouped output by 'row_id', 'AI'. You can override using the
## `.groups` argument.
```

```
bar_plot_df = rbind(plot_df %>%
  group_by(explanation, AI) %>%
  summarise(mean_payoff = mean(abs(human) - abs(hai)),
            sd_payoff = sd(abs(human) - abs(hai)),
            e = qt(0.975,df=n()-1)*sd_payoff/sqrt(n())),
  plot_df %>%
  group_by(AI) %>%
  summarise(mean_payoff = mean(abs(human) - abs(hai)),
            sd_payoff = sd(abs(human) - abs(hai)),
            e = qt(0.975,df=n()-1)*sd_payoff/sqrt(n())) %>%
    mutate(explanation = "Collapsing all explanations")) %>%
  mutate(explanation = ifelse(explanation == "both", "ILIV-SHAP and SHAP", ifelse(explanation == "shap"
  mutate(AI = ifelse(AI == "pred1", "AI1", "AI2")) %>%
  mutate(explanation = factor(explanation,
                              levels = c("ILIV-SHAP and SHAP", "SHAP", "None", "Collapsing all explanations")
  arrange(explanation, AI)
```

```
## `summarise()` has grouped output by 'explanation'. You can override using the
## `.groups` argument.
```

```
sample_size_df = rbind(filtered_exp_df %>% mutate(explanation = factor(explanation, levels = c("both",
                       filtered_exp_df %>% group_by(AI) %>% summarise(n = n()) %>% mutate(explanation =
```

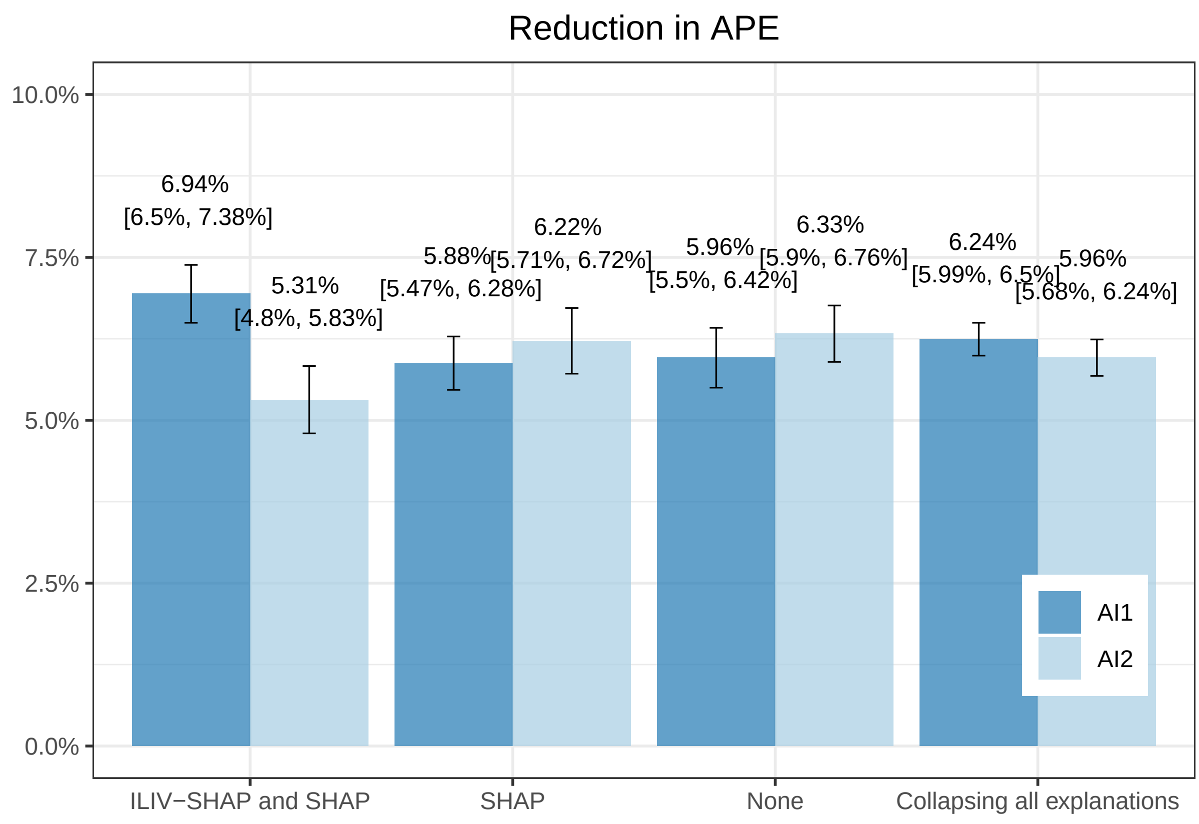
```
## `summarise()` has grouped output by 'explanation'. You can override using the
## `.groups` argument.
```

```
dev_d = 0.21
sample_annotation_df = data.frame(x = rep(1:4, each = 2) + rep(c(-dev_d, dev_d), 4),
                                  y = 0.01,
                                  label = paste("N = ", sample_size_df$n))

annotation_df = data.frame(x = rep(1:4, each = 2) + rep(c(-dev_d, dev_d), 4),
                           y = bar_plot_df$mean_payoff + bar_plot_df$e + 0.01,
                           label = paste(paste(round(bar_plot_df$mean_payoff, digits = 4) * 100, "%", s
                                         "\n [",
                                         paste(round(bar_plot_df$mean_payoff - bar_plot_df$e, digits =
                                         ", ",
                                         paste(round(bar_plot_df$mean_payoff + bar_plot_df$e, digits =
                                         "]", sep=""))
bar_plot_df %>%
  ggplot() +
  geom_bar(aes(y = mean_payoff, x = explanation, fill = AI), stat = "identity", color = NA,
```

```r
            position=position_dodge(), alpha = .7) +
  geom_errorbar(
    aes(ymin = mean_payoff - e, ymax = mean_payoff + e, x = explanation, group =  AI),
    position = position_dodge(width = 0.9),
    width = 0.1, size=0.3
  ) +
  scale_y_continuous(labels = percent, limits = c(0, 0.1)) +
  scale_fill_manual(values = c("#1f78b4", "#a6cee3")) +
  labs(title = "Reduction in APE", y = "", x = "") +
    theme_bw() +
   theme(legend.position=c(0.9, 0.2), legend.title = element_blank(), plot.title = element_text(hjust =
  annotate("text",
           x = annotation_df$x,
           y = annotation_df$y,
           label = annotation_df$label,
           size = 3)
```

```
## Warning: A numeric `legend.position` argument in `theme()` was deprecated in ggplot2
## 3.5.0.
## i Please use the `legend.position.inside` argument of `theme()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
# annotate("text",
#          x = sample_annotation_df$x,
```

```
#           y = sample_annotation_df$y,
#           label = sample_annotation_df$label,
#           size = 3)
```

```
ggsave("./exp_result.pdf", unit = "in", width = 12 * 0.65, height = 6 * 0.65)
```

Calculate the mean and CI of the reduction in APE

```
rbind(plot_df %>%
  group_by(explanation, AI) %>%
  summarise(mean_payoff = mean(abs(human) - abs(hai)),
            sd_payoff = sd(abs(human) - abs(hai)),
            e = qt(0.975,df=n()-1)*sd_payoff/sqrt(n())) %>%
    mutate(low_CI95 = mean_payoff - e,
           high_CI95 = mean_payoff + e),
  plot_df %>%
  group_by(AI) %>%
  summarise(mean_payoff = mean(abs(human) - abs(hai)),
            sd_payoff = sd(abs(human) - abs(hai)),
            e = qt(0.975,df=n()-1)*sd_payoff/sqrt(n())) %>%
    mutate(explanation = "Collapsing all explanations") %>%
    mutate(low_CI95 = mean_payoff - e,
           high_CI95 = mean_payoff + e)) %>% select(AI, explanation, mean_payoff, low_CI95, high_CI95, s
```

```
## `summarise()` has grouped output by 'explanation'. You can override using the
## `.groups` argument.
```

```
## # A tibble: 8 x 6
## # Groups:   explanation [4]
##   AI    explanation                 mean_payoff low_CI95 high_CI95 sd_payoff
##   <chr> <chr>                             <dbl>    <dbl>     <dbl>     <dbl>
## 1 pred1 both                             0.0694   0.0650    0.0738    0.0656
## 2 pred2 both                             0.0531   0.0480    0.0583    0.0746
## 3 pred1 none                             0.0596   0.0550    0.0642    0.0673
## 4 pred2 none                             0.0633   0.0590    0.0676    0.0656
## 5 pred1 shap                             0.0588   0.0547    0.0628    0.0641
## 6 pred2 shap                             0.0622   0.0571    0.0672    0.0701
## 7 pred1 Collapsing all explanations      0.0624   0.0599    0.0650    0.0658
## 8 pred2 Collapsing all explanations      0.0596   0.0568    0.0624    0.0702
```

```
plot_df %>% ungroup() %>% summarise(sd_payoff = sd(abs(human) - abs(hai)))
```

```
## # A tibble: 1 x 1
##   sd_payoff
##       <dbl>
## 1    0.0679
```

```
plot_df %>% group_by(AI, explanation) %>% summarise(obs = n(), participant = n() / 12)
```

```
## `summarise()` has grouped output by 'AI'. You can override using the `.groups`
## argument.
```

```
## # A tibble: 6 x 4
## # Groups:   AI [2]
##   AI    explanation   obs participant
```

```
##   <chr> <chr>       <int>       <dbl>
## 1 pred1 both          840          70
## 2 pred1 none          828          69
## 3 pred1 shap          948          79
## 4 pred2 both          804          67
## 5 pred2 none          888          74
## 6 pred2 shap          744          62
```

```
pairs = list(list(c("pred1", "both"), c("pred1", "shap")),
             list(c("pred1", "both"), c("pred1", "none")),

             list(c("pred1"), c("pred2")),
             list(c("pred1", "both"), c("pred2", "both")))
plot_df = plot_df %>% mutate(reduced_ape = abs(human) - abs(hai))
pvalues = c()
for (p in pairs) {
  cond1 = p[[1]]
  cond2 = p[[2]]
  if (length(cond1) > 1) {
    data1 = (plot_df %>% filter((AI == cond1[[1]]) & (explanation == cond1[[2]])))$reduced_ape
    data2 = (plot_df %>% filter((AI == cond2[[1]]) & (explanation == cond2[[2]])))$reduced_ape
  } else {
    data1 = (plot_df %>% filter((AI == cond1[[1]])))$reduced_ape
    data2 = (plot_df %>% filter((AI == cond2[[1]])))$reduced_ape
  }
  pvalues = c(pvalues, t.test(data1, data2)$p.value)
}
pvalues = p.adjust(pvalues, method = "bonferroni")
print(pvalues)
```

```
## [1] 2.202457e-03 1.048105e-02 5.547363e-01 1.215225e-05
```